

L2A: Substitution Model, Recursion

CS1101S: Programming Methodology

Boyd Anderson

19th August, 2026

Before we start...

Where are we in CS1101S?

What is CS1101S? SICPy, Source Academy, Studios, Missions, Ed, Paths

Getting into gear

- ▶ Path P2A: out today, to complete by tomorrow
- ▶ Check-In I2A: due during the lecture
- ▶ Reflection R2: sheet out on Canvas, please prepare before your reflection session tomorrow
- ▶ Mission M2A: out today after lecture
- ▶ Quest Q2A: out today evening
- ▶ Mission M2B: out tomorrow
- ▶ Quest Q2B: out on Friday
- ▶ You cannot attend a different Studio session if you miss your own

Earning eXperience Points

- ▶ Missions (basic homework): your primary source of XP
- ▶ Quests (optional homework): supplement your XP
- ▶ Early Mission/Quest submission bonus: 75 XP within 3 days; bonus decays after that
- ▶ To have your submission graded, you must **finalize** it in Source Academy, saving alone is not enough
- ▶ Finalize before the deadline to have it graded; Missions will be **auto-finalized** at the deadline
- ▶ Avengers can “unsubmit” your submission, within reasonable limits
- ▶ If you submit less than 2 days after release of Mission/Quest, Avengers will give feedback less than 4 days after release.
- ▶ Path: XP for submission
- ▶ Early path submission: 25 XP within 1 day; bonus decays after that
- ▶ No “unsubmit” for paths

Need help?

- ▶ Ed forums:
<https://edstem.org/us/courses/101145/discussion>
- ▶ PM your Avenger
- ▶ **Forum Hours.** Every Tuesday 6:30pm-8:30pm, starting from Week 3. Venue is COM1 basement.
- ▶ **Reading Assessment 1.** Practice papers will be released next week!

Substitution model, 1.1.3 and 1.1.5

Recursion and iteration, 1.2.1

Substitution model, 1.1.3 and 1.1.5

Evaluating combinations, 1.1.3

Evaluating function application, 1.1.5

Applicative vs normal order reduction, 1.1.5

Recursion and iteration, 1.2.1

Review: Evaluating arithmetic expressions, see 1.1.3

1 + 2 * 3 + 4



Replace “eligible” subexpression with result

(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4

11

Example for substitution model 1.1.5



Problem

You need to find the cost of a circular handrail, knowing that the cost of the rail per meter is 199.95 dollars, and the radius of the circle is 2.1 meters.



Evaluation of function application (1.1.5)

```
cost_per_meter = 199.95
def circumference(radius):
    return 2 * math_pi * radius

def cost_of_circular_handrail(r):
    return cost_per_meter * circumference(r)

print(cost_of_circular_handrail(2.1))
```

```
-> 199.95 * circumference(2.1)
-> 199.95 * ((2 * 3.141592) * 2.1)
-> 199.95 * (6.283185 * 2.1)
-> 199.95 * 13.19469
-> 2638.278
```



Applicative order reduction, 1.1.5

```
def square(x):  
    return x * x  
  
def sum_of_squares(x, y):  
    return square(x) + square(y)  
  
def f(a):  
    return sum_of_squares(a + 1, a * 2)  
  
print(f(5))
```

```
-> sum_of_squares(5 + 1, 5 * 2)  
-> sum_of_squares(6, 10)  
-> square(6) + square(10)  
-> (6 * 6) + (10 * 10)  
...  
-> 136
```



Normal order reduction, 1.1.5

```
def square(x):  
    return x * x  
  
def sum_of_squares(x, y):  
    return square(x) + square(y)  
  
def f(a):  
    return sum_of_squares(a + 1, a * 2)  
  
print(f(5))
```

```
-> sum_of_squares(5 + 1, 5 * 2)  
-> square(5 + 1) + square(5 * 2)  
-> ((5 + 1) * (5 + 1)) +  
    ((5 * 2) * (5 * 2))  
...  
-> 136
```

No link? Python itself evaluates in applicative order, not the normal order shown in this trace.

Substitution model for applicative-order languages

Primitive expressions : take the value

Operator combinations : evaluate operands, apply operator

Declaration assignment : evaluate the value expression and replace the name by value

Function application : evaluate component expressions

- ▶ if function is primitive: apply the primitive function
- ▶ if function is compound: substitute argument values for parameters in body of declaration

Substitution model for runes

```
def turn_upside_down(rune):  
    return quarter_turn_right(quarter_turn_right(rune))  
  
def quarter_turn_left(rune):  
    return quarter_turn_right(turn_upside_down(rune))  
  
show(quarter_turn_left(heart))
```



Substitution model for runes

quarter_turn_left(♥)

quarter_turn_r(turn_upside_down(♥))

quarter_turn_r(quarter_turn_r(quarter_turn_r(♥)))

quarter_turn_r(quarter_turn_r(♠))

quarter_turn_r(♡)



Substitution model, 1.1.3 and 1.1.5

Recursion and iteration, 1.2.1

`stackn` and `repeat_apply`

Simple factorial and its recursive process, 1.2.1

A new primitive combination: `stack_frac`

```
stack_frac(r, heart, sail)
```

splits available bounded box such that heart occupies top fraction r of box and sail occupies remaining $1 - r$ of box

Examples

```
stack_frac(87 / 100, heart, sail)
```

splits available bounded box such that heart occupies the top 87% of box and sail occupies remaining 13% of box

Trisection of the heart

```
show(stack_frac(1 / 3, heart,  
    stack_frac(1 / 2, heart, heart)))
```



Can we define `stackn` in Python?

Trisection of the heart

```
show(stack_frac(1 / 3, heart,  
               stack_frac(1 / 2, heart, heart)))
```



Quadrisection of the heart

```
show(stack_frac(1 / 4, heart,  
               stack_frac(1 / 3, heart,  
                           stack_frac(1 / 2, heart, heart))))
```



Can we generalise this idea?

A Recursive Function, first try

```
def stackn(n, rune):  
    return stack_frac(1 / n, rune,  
                      stackn(n - 1, rune))
```



Programs precisely describe a computational process.

We need to specify each step carefully and unambiguously.

The correct version

```
def stackn(n, rune):  
    return rune if n == 1 else stack_frac(1 / n, rune,  
        stackn(n - 1, rune))
```



Observation

Solution for n computed using solution $n - 1$,
solution for $n - 1$ is computed using solution $n - 2$, ...
until we reach trivial case.

A Recipe

```
def stackn(n, rune):  
    return rune if n == 1 else stack_frac(1 / n, rune,  
        stackn(n - 1, rune))
```



Recipe for recursion

- ▶ Figure out trivial *base case*
- ▶ Assume you know how to solve problem for $n - 1$.
How can we solve problem for n ?

We call this **Wishful Thinking**. Make a wish and it will come true.

A Recipe

```
def stackn(n, rune):  
    return rune if n == 1 else stack_frac(1 / n, rune,  
        stackn(n - 1, rune))
```



Recipe for recursion

- ▶ Figure out trivial *base case*
- ▶ I **wish** that I already have the solution for a problem $n - 1$.
Given that, how can I solve problem for n ?

We call this **Wishful Thinking**. Make a wish and it will come true.

It's circles all the way down...

Let's circle back to the Source Academy!

Look for Check-In I2A in the Source Academy!

Can we define `repeat_apply` in Python?

Remember pre-defined `repeat_apply` in module `repeat`

```
show(repeat_apply(make_cross, 3, sail))  
# should lead to  
show(make_cross(make_cross(make_cross(sail))))
```



Another example

```
def square(x):  
    return x * x  
  
repeat_apply(square, 3, 2)  
# should lead to  
square(square(square(2)))
```



repeat_apply, our first version

```
def repeat_apply(pat, n, init):  
    return init if n == 0 else pat(repeat_apply(pat, n - 1,  
        init))
```

```
print(repeat_apply(square, 3, 2))
```



Recursive process

The applications of `pat` *accumulate* as result of recursive calls.

They are *deferred* operations.

repeat_apply, second version

```
def repeat_apply(pat, n, rune):  
    return rune if n == 0 else repeat_apply(pat, n - 1,  
                                             pat(rune))
```

```
print(repeat_apply(square, 3, 2))
```



Iterative process

With applicative order reduction, `pat` function is applied *before* the recursive call.

There are no deferred operations.

Another example: Factorial 1.2.1

Factorial

$$n! = n(n-1)(n-2) \cdots 1$$

Grouping

$$n! = n((n-1)(n-2) \cdots 1)$$

Replacement

$$n! = n(n-1)!$$

Remember the base case

$$\begin{aligned} n! &= 1 && \text{if } n = 0 \\ n! &= n(n-1)! && \text{if } n > 0 \end{aligned}$$

Translation into Python

Remember the base case

$$\begin{aligned}n! &= 1 && \text{if } n = 0 \\n! &= n(n-1)! && \text{if } n > 0\end{aligned}$$

Factorial in Python

```
def factorial(n):  
    return 1 if n == 0 else n * factorial(n - 1)
```



Example execution using Substitution Model

```
def factorial(n):  
    return 1 if n == 0 else n * factorial(n - 1)
```

```
factorial(4)  
4 * factorial(3)  
4 * (3 * factorial(2))  
4 * (3 * (2 * factorial(1)))  
4 * (3 * (2 * (1 * factorial(0))))  
4 * (3 * (2 * (1 * 1)))  
4 * (3 * (2 * 1))  
4 * (3 * 2)  
4 * 6  
24
```



Notice the deferred operations.

Recursive process.

A Closer look at performance

Dimensions of performance

- ▶ Time:
how long does the program run
- ▶ Space:
how much memory do we need to run the program

Time for calculating $n!$

Executed Operations

Total number of executed operations

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * (1 * factorial(0))))
4 * (3 * (2 * (1 * 1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```



Space for calculating $n!$

Deferred operations:

Number of “things to remember”

```
factorial(4)
4 * factorial(3)
4 * (3 * factorial(2))
4 * (3 * (2 * factorial(1)))
4 * (3 * (2 * (1 * factorial(0))))
4 * (3 * (2 * (1 * 1)))
4 * (3 * (2 * 1))
4 * (3 * 2)
4 * 6
24
```



Can we write an iterative factorial?

```
def factorial(n):  
    return fact_iter(1, 0, n)  
  
def fact_iter(product, counter, n):  
    return product if counter == n else fact_iter(product *  
        (counter + 1),  
            counter + 1, n)
```



Summary

- ▶ Substitution allows us to trace the evaluation of expressions
 - Applicative vs normal order reduction
- ▶ Function applications are replaced by the return expression
- ▶ Examples for recursive solutions: `stackn`, `repeat_apply`, `factorial`
- ▶ Recursive and iterative processes
- ▶ Resources for computational processes:
time and space
(more on this on Friday)